Experiment 1: Random State = 0

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 4: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
```

```
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa

Classification Accuracy: 96.66666666666667 %

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.91      1.00      0.95        10
   virginica       1.00      0.90      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Random State = 1

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 4: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor

Classification Accuracy: 96.66666666666667 %

Confusion Matrix:
[[10  0  0]
 [ 0 10  0]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.91      1.00      0.95        10
   virginica       1.00      0.90      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Random State = 125

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
```

```
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 4: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=125, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create the Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict the test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]

    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: versicolor | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
```

```
Actual: setosa | Predicted: setosa

Classification Accuracy: 90.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  8  2]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.89      0.80      0.84        10
   virginica       0.82      0.90      0.86        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

Change test_size to 0.3

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 4: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
```

```
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 105
Testing samples: 45

Actual and Predicted Classes:
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor

Classification Accuracy: 91.11111111111111 %
```

Test Size = 0.4

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 4: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

```
# Step 5: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
```

```
weighted avg       0.94      0.93      0.93        60
```

Sepal Length + Sepal Width

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Select only Sepal Length and Sepal Width
X = X[:, [0, 1]]

# Step 4: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 5: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 6: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 7: Train the classifier
model.fit(X_train, y_train)

# Step 8: Predict test data
y_pred = model.predict(X_test)

# Step 9: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 10: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 11: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 12: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 2
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
```

```
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: virginica | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa

Classification Accuracy: 70.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  5  5]
 [ 0  4  6]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.56      0.50      0.53        10
   virginica       0.55      0.60      0.57        10

    accuracy                           0.70        30
   macro avg       0.70      0.70      0.70        30
weighted avg       0.70      0.70      0.70        30
```

Petal Length + Petal Width

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Select only Petal Length and Petal Width
X = X[:, [2, 3]]

# Step 4: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 5: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 6: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 7: Train the classifier
model.fit(X_train, y_train)

# Step 8: Predict test data
y_pred = model.predict(X_test)

# Step 9: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
```

```
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 10: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 11: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 12: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 2
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa

Classification Accuracy: 96.66666666666667 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  0 10]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Sepal Length + Petal Length

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Select only Sepal Length and Petal Length
X = X[:, [0, 2]]

# Step 4: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 5: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 6: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 7: Train the classifier
model.fit(X_train, y_train)

# Step 8: Predict the test data
y_pred = model.predict(X_test)

# Step 9: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 10: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 11: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 12: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 2
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
```

```
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa

Classification Accuracy: 90.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  1]
 [ 0  2  8]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

Sepal Width + Petal Width

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Select only Sepal Width and Petal Width
X = X[:, [1, 3]]

# Step 4: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
print("Target classes:", iris.target_names)

# Step 5: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 6: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 7: Train the classifier
model.fit(X_train, y_train)

# Step 8: Predict test data
y_pred = model.predict(X_test)

# Step 9: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 10: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 11: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
# Step 12: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 2
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa

Classification Accuracy: 90.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  8  2]
 [ 0  1  9]]

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.89      0.80      0.84        10
   virginica       0.82      0.90      0.86        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

Stratify = None

```
# Step 1: Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Step 2: Load the Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Step 3: Display dataset information
print("Iris Dataset")
print("Number of samples:", len(X))
print("Number of features:", X.shape[1])
```

```
print("Target classes:", iris.target_names)

# Step 4: Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=None
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# Step 5: Create Gaussian Naive Bayes classifier
model = GaussianNB()

# Step 6: Train the classifier
model.fit(X_train, y_train)

# Step 7: Predict test data
y_pred = model.predict(X_test)

# Step 8: Display actual and predicted values
print("\nActual and Predicted Classes:")
for i in range(len(y_test)):
    actual = iris.target_names[y_test[i]]
    predicted = iris.target_names[y_pred[i]]
    print("Actual:", actual, "| Predicted:", predicted)

# Step 9: Calculate classification accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nClassification Accuracy:", accuracy * 100, "%")

# Step 10: Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Step 11: Display classification report
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=iris.target_names
))
```

```
Iris Dataset
Number of samples: 150
Number of features: 4
Target classes: ['setosa' 'versicolor' 'virginica']

Training samples: 120
Testing samples: 30

Actual and Predicted Classes:
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: versicolor | Predicted: versicolor
Actual: versicolor | Predicted: versicolor
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: virginica | Predicted: virginica
Actual: setosa | Predicted: setosa
Actual: setosa | Predicted: setosa

Classification Accuracy: 100.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

```
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      1.00      1.00         9
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```
import pandas as pd

# Load and display the saved results to understand all experiments conducted
try:
    df_results = pd.read_csv('/content/Naive_Bayes_Lab8_Results.csv')
    display(df_results)
except Exception as e:
    # Fallback to display the local results_df variable if exists
    if 'results_df' in globals():
        display(results_df)
    else:
        print("Could not find a results DataFrame. Listing local environment details.")
```

```
Could not find a results DataFrame. Listing local environment details.
```